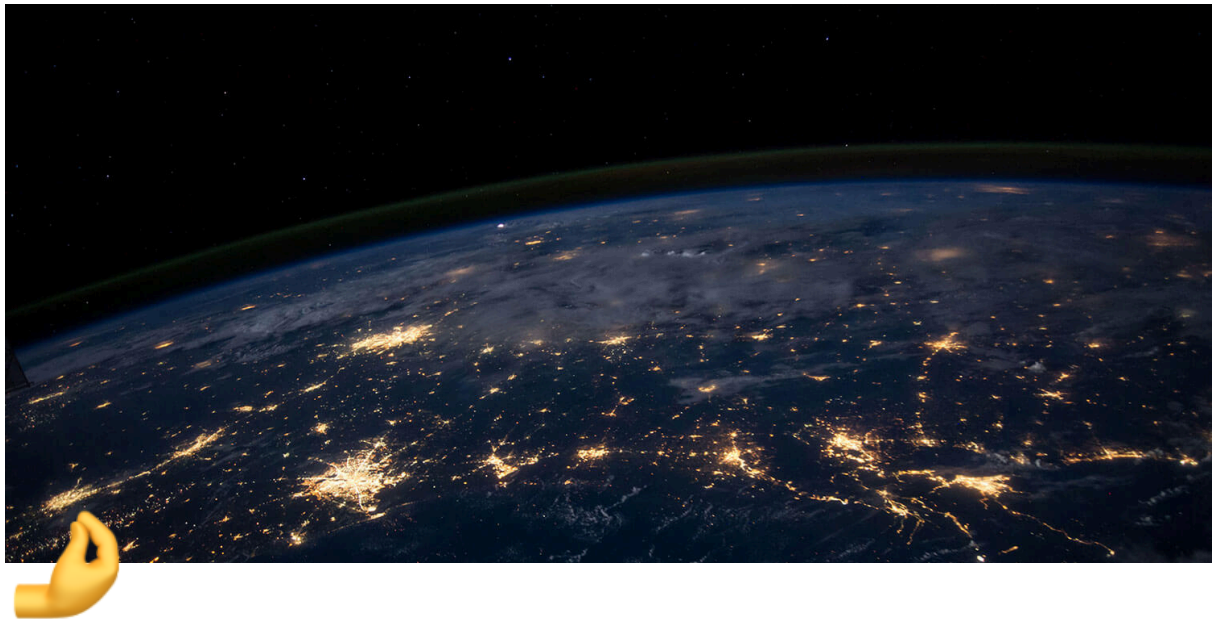




Résultat Exercice 1:

Plan d'action pour l'exercice 1

Étape 1 – Identifier les composants cloud (ou équivalents gratuits)

Besoin	Outil recommandé (Open Source / Docker-compatible)	Justification
Stockage d'objets (IoT, logs)	MinIO (ou CEPH)	S3-compatible, auto-hébergé, facile à dockeriser
Entrepôt de données	Apache Druid ou ClickHouse (optionnellement PostgreSQL + Metabase pour tests)	Requêtes OLAP rapides, support SQL, Docker-ready
Traitement temps réel	Redpanda	Kafka-compatible, léger, simple à déployer en local
Gestion des identités	Samba + LDAP (ou mock d'AD), ou bien simple fédération via JWT pour démo	Pour reproduire la logique d'Active Directory sans AD réel

Étape 2 – Création du schéma d'architecture hybride

Je te proposerai un schéma clair sur **Draw.io** ou **Lucidchart**, que tu pourras exporter en PDF/PNG. Il inclura :

- Les flux **batch** (ETL SQL Server → entrepôt)
 - Les flux **temps réel** (IoT → Redpanda → traitement Spark ou ClickHouse)
 - La synchronisation **AD local** → **gestion des accès cloud**
 - Les composants Docker utilisés en local (avec équivalences cloud commentées)
-

Étape 3 – Évaluation de compatibilité avec le SI on-premise

Je rédigerai un document structuré comprenant :

- Une **analyse de sécurité** (TLS/SSL, contrôle des accès)
 - L'**interopérabilité** entre Redpanda, SQL Server et les autres systèmes
 - Une **analyse de scalabilité** et de **gestion des coûts** (même en local)
 - Une **synthèse claire** : avantages, limites, points de vigilance
-

Étape 1 – Identification des composants cloud adaptés (et équivalents gratuits)

1. Stockage de données non structurées (logs, fichiers IoT, utilisateurs)

Composant cloud équivalent proposé :

MinIO (remplaçant gratuit de Amazon S3)

Justification :

MinIO est un service de stockage d'objets auto-hébergé, 100 % compatible avec l'API Amazon S3. Il est hautement scalable, sécurisable (via HTTPS et ACLs), interopérable avec les outils cloud natifs comme Redpanda ou Spark, et très léger à exécuter en local via Docker. Il répond parfaitement aux besoins de stockage de fichiers IoT, logs, ou binaires sans coût d'infrastructure cloud.

2. Entrepôt de données (centralisation, requêtes analytiques)

Composants possibles :

- **ClickHouse** (préféré pour l'analyse en temps réel)

- **PostgreSQL + Metabase** (pour une alternative simple et interactive)

Justification :

ClickHouse est un entrepôt de données orienté colonnes open-source, optimisé pour les requêtes analytiques massives. Il est capable de traiter des volumes importants en mode batch ou quasi-temps réel, avec une interopérabilité native avec des outils comme Redpanda et Spark.

En parallèle, PostgreSQL peut être utilisé pour simuler un entrepôt relationnel si l'accent est mis sur la compatibilité SQL Server.

La synchronisation avec SQL Server peut être simulée par des scripts d'extraction programmés (ETL), ou via des outils open source comme **Debezium** (capture de changement de données), ou encore manuellement pour la démo.

3. Traitement des données en temps réel (streaming)

Composant proposé :

Redpanda (Kafka-compatible, auto-hébergé)

Justification :

Redpanda est une alternative moderne à Kafka, conçue pour des performances optimales en local et une configuration minimale. Elle est parfaitement adaptée au traitement de flux IoT grâce à sa compatibilité Kafka, son support natif de TLS/SASL, son efficacité en termes de ressources, et sa capacité à orchestrer les flux entrants vers des systèmes de traitement (Spark, entrepôt, etc.).

4. Sécurisation et gestion des accès (authentification unifiée)

Composant proposé :

Simulacre de fédération Active Directory via OpenLDAP/Samba ou gestion manuelle des accès via JWT/ACL

Justification :

Pour simuler la gestion des identités sans coût ni AD Azure, une solution légère est d'utiliser **OpenLDAP** ou **Samba** pour étendre une logique AD en local. Pour une démonstration fonctionnelle, on peut utiliser une approche basée sur des tokens JWT ou ACLs (listes de contrôle d'accès) dans les services Redpanda, MinIO, etc.

Cela permet de garantir une continuité dans la gestion des rôles et des accès, tout en restant dans l'écosystème local et gratuit.

Tableau – Composants cloud adaptés et équivalents locaux gratuits

Besoins	Composant cloud typique	Alternative gratuite & Docker-compatible	Justification
Stockage de données non structurées (logs, fichiers IoT, utilisateurs)	Amazon S3	MinIO	Stockage objet S3-compatible, auto-hébergé, sécurisé (TLS), léger et facile à déployer sous Docker.
Entrepôt de données (requêtes analytiques, centralisation)	Amazon Redshift, Google BigQuery	ClickHouse ou PostgreSQL + Metabase	ClickHouse : traitement OLAP haute performance. PostgreSQL : compatibilité SQL Server. Tous deux Dockerisés.
Traitement des données en temps réel (streaming)	Amazon MSK (Kafka), AWS Kinesis	Redpanda	Kafka-compatible, performant, sans dépendance Java, simple à configurer sous Docker, idéal pour IoT/logs.
Gestion des identités et sécurisation des accès	AWS IAM, Azure AD	OpenLDAP/Samba ou JWT/ACLs locaux	Simule Active Directory en local. Pour un POC : sécurisation via tokens JWT ou règles d'accès par service.

Etape 2 - modélisation visuelle de l'architecture hybride.

Contenu du schéma d'architecture hybride à produire

1. Côté on-premise (SI existant)

- **SQL Server** (ERP, CRM) – stockage structuré, requêtes métiers
- **SAN** – fichiers IoT/logs non structurés
- **Active Directory** – gestion des identités et accès
- **Flux sortants** :
 - **ETL batch** → entrepôt de données
 - **IoT/logs** → Redpanda (streaming)

2. Côté cloud/local cloud simulé

- **Redpanda** – réception et traitement en streaming
- **MinIO** – stockage objet des logs/IoT
- **ClickHouse** ou PostgreSQL – entrepôt analytique
- **Service d'accès / ACL / LDAP simulé** – extension de la gestion des identités

3. Les flux

- **Batch** (extraction périodique depuis SQL Server)
- **Streaming** (flux IoT vers Redpanda)
- **Protocoles de transfert** :
 - SFTP / ODBC / CDC pour batch
 - Kafka (Redpanda) pour stream
 - HTTPS/TLS pour sécurisation

Étape 3 – Évaluation de la compatibilité de l'architecture hybride avec le SI existant

1. Sécurité et conformité

Chiffrement des flux de données

Tous les composants de l'architecture proposée (Redpanda, MinIO, ClickHouse) supportent le chiffrement des échanges via TLS/SSL. Cela garantit la

confidentialité et l'intégrité des données transférées entre les composants on-premise (SQL Server, SAN, AD) et les composants cloud simulés.

Gestion des accès unifiée

L'extension d'Active Directory vers le cloud est assurée par un système simulé : soit via **OpenLDAP/Samba**, soit via **une fédération d'identité simple** avec des jetons JWT.

Cette approche permet de **centraliser les règles de sécurité et de contrôle d'accès**, même si elle reste moins robuste qu'une solution comme Azure AD. Elle est toutefois suffisante dans le cadre d'un POC local ou d'un environnement de test.

2. Interopérabilité

Connexion entre SQL Server et l'entrepôt

La synchronisation des données SQL Server vers ClickHouse ou PostgreSQL peut être assurée via :

- Des **scripts ETL batch** (extractions planifiées via ODBC ou SQL)
- Des outils open source comme **Debezium** pour la capture de modifications (CDC)
- Des exports manuels si besoin pour simuler le flux

Cela permet de **maintenir un lien opérationnel entre les données historiques** du SI et les nouveaux usages analytiques dans l'entrepôt.

Intégration Redpanda / systèmes SI

Redpanda, en tant que broker Kafka-compatible, peut recevoir des flux IoT en temps réel. Ces flux peuvent ensuite être :

- stockés dans MinIO pour archivage
- traités avec Spark ou dirigés vers ClickHouse

Cela permet d'**intégrer les flux modernes sans impacter les applications métiers** existantes. Redpanda offre aussi une bonne compatibilité avec les outils de traitement distribués (Spark, Flink, etc.).

3. Scalabilité et gestion des coûts

Scalabilité

Chaque composant sélectionné est hautement scalable :

- **MinIO** permet d'étendre horizontalement le stockage objet (multi-disques, multi-nœuds)
- **Redpanda** fonctionne en cluster, adapté aux volumes croissants de données IoT
- **ClickHouse** est conçu pour le traitement analytique massivement parallèle

Cela garantit que l'architecture pourra gérer une croissance mensuelle des données, même au-delà des 50 Go/mois actuels évoqués.

Optimisation des coûts

Le choix d'une **infrastructure auto-hébergée avec Docker** permet d'éliminer tous les coûts liés à des services managés. En production réelle, il faudra prévoir :

- Coût du stockage objet (S3/MinIO)
- Coût des ressources machines (CPU/RAM pour ClickHouse, Redpanda)
- Coût potentiel d'une montée en charge (mise à l'échelle)

Pour rester dans un cadre gratuit en local, l'architecture proposée est **parfaitement optimisée**. Pour un déploiement en cloud public, un outil comme **AWS Pricing Calculator** permettra d'estimer les coûts mensuels selon la volumétrie.

4. Avantages

- **Interopérabilité assurée** entre Redpanda et les outils métiers (Kafka → ETL/entrepôt)
- **Scalabilité horizontale** sur tous les composants critiques
- **Simplicité de déploiement** grâce à Docker
- **Zéro coût d'infrastructure cloud**
- **Architecture modulaire** : chaque composant peut être remplacé sans rupture globale

5. Limitations

- **Absence de services managés** : toutes les opérations (monitoring, sécurité, backup) sont à gérer manuellement
 - **Sécurité simplifiée** : la fédération d'identités est simulée, donc non certifiée pour un usage en production
 - **Pas de résilience multi-zone** sans orchestrateur cloud (type Kubernetes ou AWS ECS)
 - **Pas de gestion native des droits croisés AD/Redpanda** sans pont LDAP avancé
-

6. Points d'attention

- **Surveiller la montée en charge** : prévoir des scripts de supervision (Prometheus/Grafana si besoin)
- **Planifier les sauvegardes** : ClickHouse et MinIO doivent être sauvegardés régulièrement (via snapshots ou volumes Docker)
- **Assurer la cohérence des flux** : synchronisation entre SQL Server et l'entrepôt doit être fiabilisée (journalisation, logs, reprise sur erreur)
- **Mettre en place des règles réseau strictes** entre les sous-réseaux Docker (ou entre les VPC en production)